

NT230

CƠ CHẾ HOẠT ĐỘNG CỦA MÃ ĐỘC

GV: TS. Phan Thế Duy

Trường Đại học Công nghệ Thông tin
Đại học Quốc gia Thành phố Hồ Chí Minh

BUỔI 02 · TẬP THỰC THI VÀ NỘI BỘ WINDOWS





Buổi 02

Tập thực thi và nội bộ Windows

Câu hỏi xuyên suốt

Windows biến một tệp PE thành mã đang thực thi như thế nào, và vì sao cần hiểu điều đó trước khi phân tích lây nhiễm?

- 90 phút trên lớp; mẫu PE lành được dùng để quan sát.
- Đầu ra: sơ đồ file–memory và kết luận có giới hạn.

Nội dung chính

- 1 Định hướng và bối cảnh
- 2 Cấu trúc Portable Executable
- 3 Địa chỉ và ánh xạ file–memory
- 4 Windows loader và mô hình thực thi
- 5 Đối chiếu PE–ELF
- 6 Windows Internals và DLL
- 7 Tổng kết và Exit Ticket

1

Định hướng và bối cảnh

Câu hỏi trọng tâm

Một tệp Windows trở thành mã đang chạy qua những bước nào?

Mục tiêu của Buổi 2

- Mô tả cấu trúc PE và vai trò các thành phần chính.
- Phân biệt file offset, RVA, VA và ánh xạ file—memory.
- Giải thích loader, process, thread và memory map.
- Nhận diện dấu hiệu cần điều tra, không kết luận từ một chỉ báo.
- Chuẩn bị mô hình chuẩn cho Bài 3: đối chiếu trạng thái trước khi bị can thiệp.

Tình huống mở đầu

Dữ kiện

Một .exe giả dạng công cụ học tập nạp thêm DLL và tạo nhiều vùng nhớ khi chạy.

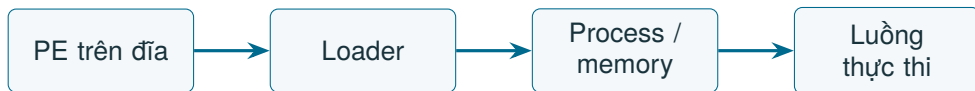
Cá nhân → cặp đôi · 3 phút

Nêu ba thông tin cần kiểm tra trong tệp và ba thông tin cần kiểm tra trong process. Chưa cần kết luận có mã độc.

Cầu nối

Muốn nhận diện sự can thiệp ở Bài 3, trước hết phải biết trạng thái PE và quá trình nạp bình thường.

Sợi dây liên kết với Bài 3



Bài 2

Xây dựng trạng thái chuẩn và định vị bằng chứng.

Bài 3

Dùng trạng thái chuẩn để phân tích tệp hoặc tiến trình bị can thiệp.

Vì sao phải học PE trước khi học lây nhiễm?

- Mã độc phải tồn tại trong tệp, được nạp hoặc được thực thi trong một tiến trình.
- Mỗi vị trí mã, dữ liệu và điểm bắt đầu đều có biểu diễn trong PE hoặc memory map.
- Nếu không biết trạng thái bình thường, không thể giải thích chính xác sự khác biệt sau can thiệp.
- Bài 2 cung cấp ngôn ngữ: section, entry point, module, địa chỉ và quyền bộ nhớ.

2

Cấu trúc Portable Executable

Câu hỏi trọng tâm

PE mô tả một image Windows như thế nào?

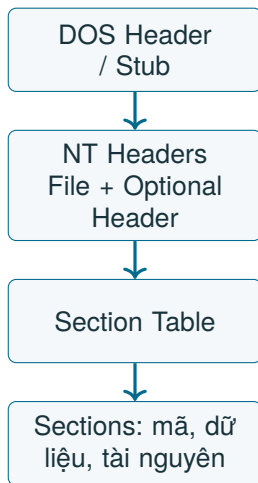
Portable Executable là gì?

Định nghĩa

PE là định dạng mô tả image thực thi của Windows, gồm metadata, vùng mã/dữ liệu và thông tin để loader đưa image vào bộ nhớ.

- Ví dụ: .exe, .dll; .sys giới thiệu ở mức khái niệm.
- PE không đồng nghĩa mã độc: phần mềm lành và mã độc đều có thể là PE.
- Phân tích PE là phân tích cấu trúc và bằng chứng, không chỉ đọc tên tệp.

Bản đồ tổng quát của một PE



PE header: những trường cần nhận diện

- MZ, PE
0
0: chữ ký định dạng.
- Machine, NumberOfSections: kiến trúc và quy mô image.
- AddressOfEntryPoint: điểm bắt đầu theo metadata.
- ImageBase, SectionAlignment, FileAlignment.
- SizeOfImage, Subsystem, DataDirectory.

Cảnh báo

Entry point là điểm bắt đầu được mô tả trong PE; không khẳng định đó là toàn bộ logic quan trọng.

Sections: mã, dữ liệu và tài nguyên

Section	Vai trò khái quát
.text	Mã máy.
.rdata	Dữ liệu chỉ đọc, chuỗi, bảng.
.data	Dữ liệu có thể ghi.
.rsrc	Tài nguyên.
.reloc	Thông tin điều chỉnh địa chỉ.

Trong phân tích mã độc

Tên section, kích thước, entropy và quyền R/W/X là tín hiệu đặt câu hỏi, không phải bằng chứng kết luận riêng lẻ.

DOS Header và DOS Stub

- Chữ ký MZ thường nằm ở đầu tệp.
- DOS Stub là phần tương thích lịch sử, không phải điểm phân tích chính của mã Windows hiện đại.
- Trường `e_lfanew` dẫn tới NT Headers.

Kỹ năng đọc

Không chỉ kiểm tra tệp bắt đầu bằng MZ; cần lần theo con trỏ tới PE signature.

COFF/File Header

- Machine cho biết kiến trúc mục tiêu, ví dụ x86 hoặc x64.
- NumberOfSections cho biết bảng section có bao nhiêu mục.
- Characteristics mô tả một số thuộc tính của image.
- Metadata thời gian chỉ hỗ trợ điều tra, không phải bằng chứng nguồn gốc tuyệt đối.

Optional Header

Vì sao gọi là Optional?

Tên gọi có tính lịch sử; với image thực thi, phần này chứa nhiều thông tin thiết yếu cho loader.

- Entry point và image base.
- Căn chỉnh trên đĩa và trong bộ nhớ.
- Kích thước image và subsystem.
- Data directories trỏ tới import, export, resource, relocation, TLS.

Data Directory

- Là bảng con trỏ/kích thước tới các cấu trúc dữ liệu quan trọng.
- Import Directory: liên kết thư viện/API.
- Base Relocation: điều chỉnh địa chỉ.
- TLS Directory: callback và dữ liệu TLS.
- Resource Directory: tài nguyên nhúng.

Cách đọc

Directory trỏ tới dữ liệu; muốn kết luận phải kiểm tra dữ liệu thực tế và bối cảnh chạy.

Characteristics và Subsystem

- Characteristics hỗ trợ nhận diện loại image và thuộc tính thực thi.
- Subsystem gợi ý môi trường giao diện, console hoặc thành phần hệ thống.
- Đây là metadata giúp công cụ phân tích trình bày đúng ngữ cảnh.
- Metadata không thay thế phân tích mã và hành vi.

Section Table đọc như thế nào?

- Mỗi hàng mô tả vị trí, kích thước và thuộc tính của một section.
- Đọc đồng thời tên, raw range, virtual range và quyền.
- Kiểm tra tính liên tục, chồng lấn và căn chỉnh.
- So sánh với một PE cùng loại để hình thành baseline.

VirtualSize và SizeOfRawData

- `SizeOfRawData`: dữ liệu section lưu trong file.
- `VirtualSize`: kích thước section khi hiện diện trong bộ nhớ.
- Hai giá trị có thể khác nhau do căn chỉnh và dữ liệu chưa khởi tạo.
- Sai khác hợp lệ của định dạng không tự động là dấu hiệu chèn mã.

Section alignment và padding

- FileAlignment căn dữ liệu trên đĩa.
- SectionAlignment căn dữ liệu trong bộ nhớ.
- Padding tạo khoảng trống giữa các vùng.
- Không nhầm padding với vùng mã hoặc dữ liệu mới.

3

Địa chỉ và ánh xạ file-memory

Câu hỏi trọng tâm

Một byte trên đĩa được tìm thấy ở đâu khi image chạy?

File offset, RVA và VA

File offset

Vị trí của dữ liệu trong tệp trên đĩa.

RVA

Địa chỉ tương đối so với image base của image.

VA

Địa chỉ ảo khi image được nạp vào không gian địa chỉ của process.

Công thức

$$VA = ImageBase + RVA$$

Từ file offset đến RVA

Công thức trong một section

$$\text{RVA} = \text{Section.VirtualAddress} + (\text{FileOffset} - \text{Section.PointerToRawData})$$

Điều kiện

File offset phải thuộc vùng raw data của section và thông tin section phải hợp lệ.

Cập đôi · 5 phút

Với VA cơ sở, VirtualAddress, PointerToRawData và một file offset cho trước, tính RVA và VA; ghi rõ từng bước.

Ví dụ tính địa chỉ

- ImageBase = 0x400000.
- Section.VirtualAddress = 0x2000.
- PointerToRawData = 0x600.
- FileOffset = 0xA00.

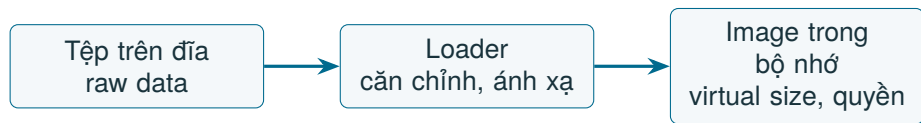
Đáp án

Offset tương đối = 0x400; RVA = 0x2400; VA = 0x402400.

Liên kết Bài 3

Muốn đối chiếu vị trí trong tệp với vùng nhớ sau can thiệp, phải phân biệt ba loại địa chỉ này.

File image và memory image



- Kích thước và vị trí trên đĩa không luôn trùng với trạng thái trong bộ nhớ.
- Đây là cơ sở để phát hiện và giải thích sai khác, không tự động chứng minh lây nhiễm.

Import và DLL

- Import thể hiện thư viện và API mà image khai báo sử dụng.
- Nhóm API có thể gợi ý thao tác tệp, process, mạng, registry hoặc bộ nhớ.
- Import cho biết khả năng/phụ thuộc, không chứng minh API chắc chắn đã được gọi.
- Cần kết hợp với disassembly, runtime và log.

Liên kết

Bài 3 sẽ dùng khái niệm DLL/module để phân tích sâu hơn sự thay đổi trong process.

Relocation và ImageBase

- Image có thể không được nạp đúng địa chỉ cơ sở kỳ vọng.
- Relocation giúp điều chỉnh tham chiếu khi địa chỉ nạp thay đổi.
- Phải phân biệt địa chỉ kỳ vọng, địa chỉ thực tế và RVA.
- ASLR là một lý do khiến địa chỉ quan sát giữa các lần chạy có thể khác nhau.

Sai lầm

Không coi mọi khác biệt địa chỉ là dấu hiệu bị sửa đổi.

TLS callback và hoạt động trước Entry Point

Khái niệm

TLS callback là hàm có thể được gọi trong các sự kiện liên quan đến module/thread, có thể xảy ra trước entry point.

- Chỉ quan sát entry point có thể bỏ sót đường thực thi.
- Có TLS callback không đồng nghĩa chắc chắn là mã độc.
- Cần ghi lại vị trí, điều kiện gọi và hành vi quan sát được.

Kiểm tra nhanh: chỉ báo hay bằng chứng?

Cá nhân · 2 phút

Một PE có section tên lạ, entropy cao và import ít. Chọn phát biểu đúng:

- A. Chắc chắn là mã độc.
- B. Chắc chắn là packer hợp lệ.
- C. Có dấu hiệu cần điều tra thêm.

Yêu cầu

Chọn một phương án và ghi thêm một bằng chứng cần thu thập.

Phân tích đáp án: C là lựa chọn đúng

Đáp án

C. Có dấu hiệu cần điều tra thêm.

- Section tên lạ có thể do compiler, packer hoặc quy ước riêng của phần mềm.
- Entropy cao có thể gợi ý dữ liệu nén hoặc mã hóa, nhưng không chứng minh mục đích độc hại.
- Import ít có thể do nạp API động, bị tối giản hoặc do đặc điểm của chương trình.
- Cần đối chiếu headers, entry point, section permissions, strings, runtime và mẫu đối chứng.

Vì sao A và B sai?

A kết luận quá mức từ chỉ báo; B cũng kết luận quá mức theo hướng ngược lại.

5

Windows loader và mô hình thực thi

Câu hỏi trọng tâm

Windows biến PE thành process như thế nào?

Chuỗi nạp PE



Đọc sơ đồ từ trái sang phải

Mỗi bước tạo ra hoặc làm rõ một loại bằng chứng: metadata, địa chỉ, module, quyền và luồng thực thi.

Process, thread và module

- Process: không gian địa chỉ và tài nguyên của chương trình đang chạy.
- Thread: luồng thực thi trong process.
- Module: image hoặc DLL được nạp vào process.
- Memory map: các vùng nhớ, địa chỉ và quyền tương ứng.

Vì sao cần hiểu?

Bài 3 cần xác định tệp nào, process nào, module nào và vùng nhớ nào đã thay đổi.

Quyền bộ nhớ và dấu hiệu cần xem xét

Quyền	Ý nghĩa khái quát
R	Đọc.
W	Ghi.
X	Thực thi.
RWX	Đọc–ghi–thực thi; cần giải thích theo ngữ cảnh.

Nguyên tắc

Quyền bất thường là điểm bắt đầu điều tra; phải đối chiếu module, thời điểm, API và hành vi.

Demo quan sát PE lành

- Mở mẫu trong PE-bear/PEStudio: headers, sections, imports, entry point.
- Dùng Ghidra để định vị entry point và vùng mã.
- Chạy trong môi trường cô lập; quan sát process/module/memory map ở mức an toàn.
- Ghi hash, công cụ, phiên bản và cấu hình.

Sản phẩm

Bảng quan sát và sơ đồ file–memory; chưa chỉnh sửa hoặc phát tán mẫu.

6

Đối chiếu PE–ELF

Câu hỏi trọng tâm

Cùng một câu hỏi phân tích có áp dụng cho mọi nền tảng không?

PE và ELF: điểm chung và khác biệt

	PE	ELF
Nền tảng	Windows	Linux/Unix phổ biến
Metadata	Header/section/directory	Header/section/segment
Thực thi	Windows loader	ELF loader/linker

Mục tiêu

Nhận diện bối cảnh và tránh áp dụng nhầm thuật ngữ; không phân tích ELF chuyên sâu trong bài này.

Luyện tập tích hợp

Nhóm 2–3 · 10 phút

Từ bảng PE mẫu, hãy xác định: kiến trúc, số section, entry point, image base, một import đáng chú ý và một vùng cần giải thích thêm.

Sản phẩm

Một bảng quan sát, một sơ đồ file–memory và ba câu kết luận theo mẫu: dữ kiện → suy luận → giới hạn.

Vai trò

Người phân tích · người kiểm chứng · người phản biện.

Lỗi thường gặp cần sửa

- Nhầm file offset với RVA hoặc VA.
- Cho rằng section tên lạ chắc chắn là mã độc.
- Cho rằng import chứng minh hành vi đã xảy ra.
- Cho rằng entry point là vị trí duy nhất cần phân tích.
- Đồng nhất file image với memory image.
- Suy diễn lây nhiễm chỉ từ một ảnh chụp công cụ.

Header và điểm bắt đầu thực thi

Câu hỏi	PE	ELF
Chữ ký/loại image	PE signature, File/Optional Header	ELF magic, ELF Header
Kiến trúc	Machine trong COFF Header	e_machine trong ELF Header
Điểm bắt đầu	AddressOfEntryPoint	e_entry
Địa chỉ nạp	ImageBase, RVA/VA	Load bias, virtual address

Section của PE và segment của ELF

PE

Section Table mô tả các vùng mã, dữ liệu, tài nguyên và relocation; section thường được dùng để đọc cấu trúc image.

ELF

Program Header mô tả các segment cần cho quá trình nạp; section thường phục vụ liên kết và phân tích file.

Bài học

Không mặc định một section PE tương ứng một-một với một segment ELF.

Liên kết thư viện trên hai nền tảng

- PE dùng import/export và IAT để nối image với DLL/API.
- ELF dùng dynamic section, symbol table, relocation và cơ chế linker/loader tương ứng.
- Cả hai đều cần phân biệt: khai báo phụ thuộc, địa chỉ được resolve và hành vi thực tế.
- Không dùng thuật ngữ IAT để mô tả mọi cơ chế liên kết trên ELF.

Địa chỉ và relocation: cùng câu hỏi, khác biểu diễn

- Câu hỏi chung: image được nạp ở đâu và địa chỉ nào cần điều chỉnh?
- PE: dùng ImageBase, RVA, VA và relocation entries.
- ELF: dùng virtual address, load bias và relocation của linker/loader.
- Khi so sánh, quy đổi về quan hệ “file location → image location → runtime address”.

Quy trình đối chiếu không phụ thuộc nền tảng

- 1 Xác định nền tảng, kiến trúc và loại image.
- 2 Đọc header và entry point theo đúng định dạng.
- 3 Xác định vùng mã/dữ liệu và cách ánh xạ.
- 4 Xem dependency, symbol và relocation.
- 5 Đối chiếu file image với memory image.
- 6 Ghi rõ công thức, công cụ và giới hạn suy luận.

Luyện tập tích hợp: yêu cầu sản phẩm

- Bảng PE: kiến trúc, số section, entry point, image base, import.
- Bảng đối chiếu: một khái niệm chung và tên biểu diễn tương ứng trên ELF.
- Sơ đồ file–memory cho PE.
- Ba kết luận theo mẫu: dữ kiện → suy luận → giới hạn.
- Phân công: người phân tích, kiểm chứng, phản biện.

7

Windows Internals và DLL

Câu hỏi trọng tâm

Một số cơ chế liên quan đến việc nạp và thực thi DLL là gì?

Windows Internals: phạm vi cần nắm

- Process cung cấp address space và tài nguyên cho chương trình.
- Thread là đơn vị thực thi trong process.
- Image chính và DLL được biểu diễn như các module.
- Loader nối PE trên đĩa với module trong bộ nhớ.

Mục tiêu của phần này

Hiểu cơ chế nạp và liên kết DLL để có baseline trước khi phân tích can thiệp vào process ở Bài 3.

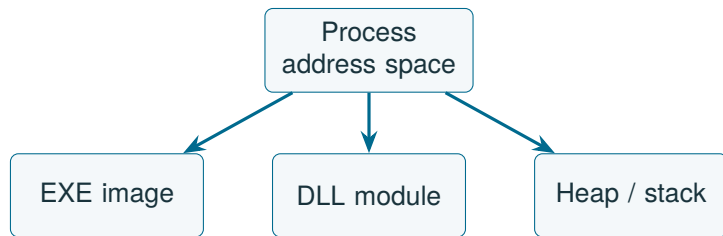
EXE image và DLL image

	EXE	DLL
Vai trò	Image chính để tạo process	Module được nạp vào process
Entry point	Điểm bắt đầu process	Điểm vào DLL khi có sự kiện module/thread
Không gian nhớ	Process sở hữu image	Chia sẻ address space của process
Phụ thuộc	Có thể import DLL	Có thể import và export hàm

Điểm cần nhớ

DLL không tự động là mã độc; cần xét nguồn, đường dẫn, chữ ký, quan hệ module và hành vi.

Image, module và section trong memory



Phân tích

Cần xác định vùng đang quan sát thuộc image module hay private memory; hai loại này có ý nghĩa khác nhau.

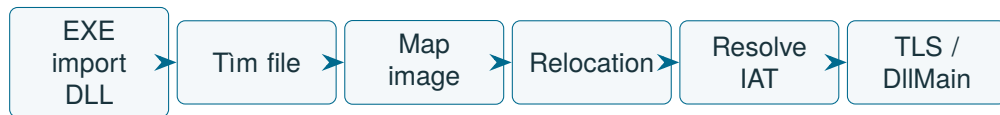
Khi nào DLL được nạp?

- DLL có thể là dependency được khai báo trong Import Directory.
- DLL cũng có thể được yêu cầu trong runtime bởi cơ chế nạp động.
- Loader có thể nạp dependency của DLL theo chuỗi nhiều tầng.
- Một module chỉ xuất hiện trong process sau khi được ánh xạ và khởi tạo phù hợp.

Giới hạn

Không thấy tên DLL trong import table không chứng minh DLL không bao giờ được sử dụng.

Chuỗi nạp DLL: nhìn từ trái sang phải



Ý nghĩa

Mỗi bước tạo loại bằng chứng khác nhau: đường dẫn, module base, relocation, địa chỉ API và sự kiện khởi tạo.

Bước 1: tìm DLL cần nạp

- Loader xác định tên DLL và các dependency.
- Việc tìm file phụ thuộc tên, đường dẫn, cấu hình và ngữ cảnh process.
- Khác biệt 32-bit/64-bit có thể dẫn tới DLL khác nhau.
- Nhà phân tích ghi lại đường dẫn tuyệt đối, hash, chữ ký và kiến trúc.

Bằng chứng

Import table cho biết yêu cầu; filesystem và module list cho biết file thực tế được dùng.

DLL search order và nguy cơ nạp nhầm

Khái niệm

Search order là quy tắc hệ thống dùng để tìm file DLL khi tên file chưa xác định bằng đường dẫn tuyệt đối.

- Thứ tự tìm kiếm phụ thuộc cơ chế gọi và cấu hình Windows.
- Một DLL cùng tên ở vị trí ưu tiên không phù hợp có thể bị chọn nhầm.
- Đây là nền tảng để hiểu DLL search-order hijacking ở mức nhận diện.

Phạm vi

Không thực hiện kỹ thuật chiếm quyền; chỉ phân tích đường dẫn và bằng chứng nạp.

Bước 2: map DLL vào address space

- Loader tạo các mapping tương ứng với sections của DLL.
- Image base ưu tiên có thể khác địa chỉ nạp thực tế.
- Mỗi section nhận kích thước và quyền phù hợp.
- Module base giúp quy đổi RVA của DLL sang VA trong process.

Câu hỏi phân tích

Địa chỉ module có thuộc file được xác minh không? Vùng memory có khớp sections và quyền dự kiến không?

Bước 3: relocation của DLL

- 1 DLL có preferred ImageBase.
- 2 Nếu địa chỉ đã được sử dụng, DLL được nạp ở base khác.
- 3 Relocation entries điều chỉnh tham chiếu cần thiết.
- 4 Nhà phân tích dùng module base + RVA để xác định VA.

Tránh nhầm

Hai process có thể có VA khác nhưng cùng một RVA và cùng một file DLL.

Bước 4: resolve symbol và API

- DLL cung cấp hàm qua Export Directory.
- EXE/DLL yêu cầu hàm qua Import Directory.
- Loader ghép yêu cầu với địa chỉ export tương ứng.
- Địa chỉ được ghi vào vùng liên quan tới Import Address Table.
- Import theo tên và theo ordinal có cách nhận diện khác nhau.

IAT và vai trò trong phân tích

Import Address Table

IAT chứa các địa chỉ mà image dùng để gọi hàm được cung cấp bởi module khác sau khi liên kết.

- Trước resolve, entry có thể chứa là địa chỉ hàm cuối cùng.
- Sau resolve, lệnh gọi gián tiếp có thể đi qua IAT.
- IAT giúp nối disassembly với API và module cung cấp API.
- IAT không phải nhật ký số lần API được gọi.

Export theo tên và theo ordinal

- Export theo tên dễ đọc và đối chiếu hơn.
- Export theo ordinal dùng chỉ số; cần tra bảng export để xác định hàm.
- Forwarded export có thể chuyển tiếp yêu cầu sang module khác.
- Tên API bị làm rối hoặc thiếu import không loại trừ khả năng sử dụng chức năng tương ứng.

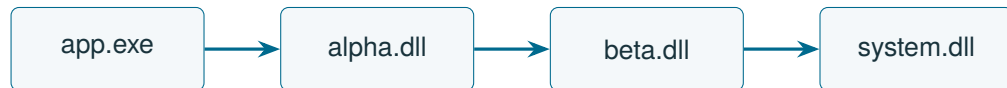
Bước 5: TLS callback và DllMain

- TLS callback có thể nhận sự kiện liên quan đến module/thread.
- DLL có thể có logic khởi tạo khi được nạp hoặc gỡ.
- DllMain là điểm khởi tạo theo quy ước của DLL, không phải toàn bộ hành vi.
- Cần ghi nhận thứ tự, điều kiện và hành vi quan sát được.

Nguyên tắc

Không suy luận độc hại chỉ từ sự tồn tại của TLS callback hoặc DllMain.

DLL dependency chain



- Một hành vi quan sát được có thể liên quan tới nhiều tầng dependency.
- Cần ghi module trực tiếp và module gián tiếp; không gán hành vi cho DLL đầu tiên nhìn thấy.

PEB và danh sách module ở mức khái niệm

PEB

Process Environment Block là cấu trúc dữ liệu gắn với process, chứa thông tin môi trường và con trỏ tới dữ liệu loader.

Loader data

Thông tin loader hỗ trợ theo dõi các module đã biết, địa chỉ cơ sở và đường dẫn ở mức runtime.

Phạm vi

Sinh viên cần hiểu ý nghĩa đối chiếu; không yêu cầu thao tác trực tiếp trên cấu trúc nội bộ trong bài này.

Ba nguồn để kiểm tra module

Nguồn	Câu hỏi
PE trên đĩa	File có headers, export/import và chữ ký gì?
Module list	Module nào hiện diện, ở base nào, đường dẫn nào?
Memory map	Vùng nhớ có khớp image sections và quyền không?

Kết luận mạnh

Chỉ khi các nguồn hỗ trợ lẫn nhau mới nên nâng mức tin cậy của giả thuyết.

Image-backed và private memory

- Image-backed memory thường liên hệ với file image hoặc DLL trên đĩa.
- Private memory được cấp phát cho process và không nhất thiết có file tương ứng.
- Cần xem protection, allocation type, địa chỉ và thời điểm tạo.
- Một vùng private executable là tín hiệu cần điều tra, chưa đủ chứng minh injection.

Phân biệt DLL loading và DLL injection

	Loading hợp lệ	Nghi vấn can thiệp
Nguồn	Dependency hoặc yêu cầu ứng dụng	Đường dẫn/module không giải thích được
Địa chỉ	Khớp image và relocation	Khớp kém hoặc vùng bất thường
Bằng chứng	Import/module/load event	Process access, memory và thread context
Kết luận	Được loader xử lý	Chỉ là giả thuyết cần kiểm chứng

Vì sao không kết luận injection từ một DLL?

- DLL có thể được ứng dụng nạp theo thiết kế.
- DLL có thể là dependency gián tiếp.
- Module list không cho biết đầy đủ cách module được đưa vào process.
- Cần kết hợp process access, thread, memory region, đường dẫn và timeline.

Cầu nối Bài 3

Bài 3 sẽ phân tích các cơ chế can thiệp; Bài 2 chỉ xây dựng baseline để phân biệt chúng với loading thông thường.

Bài tập: đọc dependency và IAT

Nhóm 2–3 · 8 phút

Từ một bảng import/export được cung cấp, hãy dựng dependency chain, xác định module cung cấp một API và nêu một bằng chứng runtime cần thu thêm.

Sản phẩm

Sơ đồ module, bảng API–DLL và một giới hạn kết luận.

Bài tập: giải thích module bất thường

Dữ kiện

Một DLL nằm trong thư mục ứng dụng, cùng tên với DLL hệ thống; process đã nạp DLL tại địa chỉ khác ImageBase.

Cặp đôi · 5 phút

Nêu ba giả thuyết có thể có và dữ liệu cần thu để phân biệt chúng.

Đáp án định hướng

- Khác ImageBase có thể là relocation/ASLR hợp lệ.
- Đường dẫn ứng dụng có thể hợp lệ hoặc cần xem xét search-order.
- Cần hash, chữ ký, kiến trúc, export, process context và timeline load.
- Chưa đủ dữ kiện để kết luận DLL bị thay thế hoặc được nạp độc hại.

Checklist phân tích DLL

- 1 Tên, đường dẫn, hash, chữ ký và kiến trúc.
- 2 EXE/DLL nào yêu cầu module? Trực tiếp hay gián tiếp?
- 3 Base address, RVA, sections và quyền memory.
- 4 Import/export, IAT và symbol resolution.
- 5 TLS/DllMain, process/thread và load timeline.
- 6 Đối chiếu PE trên đĩa với module trong bộ nhớ.

Tổng kết phần Windows Internals

- Loader tìm, map, relocate, resolve và khởi tạo DLL.
- IAT nối lời gọi trong image với API được export.
- PEB/LDR và memory map cung cấp góc nhìn runtime về module.
- DLL loading hợp lệ khác với DLL injection; không suy luận từ một dấu hiệu.
- Các baseline này là nền tảng trực tiếp để học Bài 3.

8

Tổng kết và Exit Ticket

Câu hỏi trọng tâm

Ta đã có mô hình chuẩn nào để bước vào Bài 3?

Tổng kết Buổi 2

- PE mô tả image Windows và các vùng mã/dữ liệu.
- Header, section, import, relocation, TLS cung cấp các điểm quan sát khác nhau.
- Loader ánh xạ file image vào process và memory map.
- Offset, RVA, VA giúp nối bằng chứng trên đĩa với bằng chứng trong bộ nhớ.
- Dấu hiệu bất thường cần được kiểm chứng bằng nhiều nguồn.

Exit ticket cá nhân: tổng kết Buổi 2

Trả lời ba câu hỏi

- ❶ **Q1:** Một section có quyền RWX và entropy cao. Vì sao đây mới là dấu hiệu cần điều tra, chưa đủ để kết luận PE là mã độc?
- ❷ **Q2:** Một EXE có import tới một DLL, nhưng DLL được nạp ở ImageBase khác với giá trị ưu tiên. Hiện tượng này có thể được giải thích như thế nào và cần kiểm tra bằng chứng nào?
- ❸ **Q3:** Vì sao phải hiểu PE-loader-memory map trước khi phân tích cơ chế lây nhiễm ở Bài 3?

Sản phẩm: Trả lời ngắn gọn, tối đa 150 từ; ghi họ tên và mã học viên.

Đáp án cốt lõi

Câu trả lời cần phân biệt chỉ báo với bằng chứng, giải thích được relocation/ASLR và nêu vai trò của baseline PE-loader-memory khi đối chiếu sự can thiệp.

Đáp án gợi ý: Exit ticket Buổi 2

❶ Q1 – Chỉ báo và bằng chứng

Quyền RWX và entropy cao có thể xuất hiện ở vùng mã được đóng gói, dữ liệu nén hoặc chương trình có thiết kế đặc biệt. Cần kiểm tra section, entry point, imports, nguồn file, runtime và mẫu đối chứng trước khi kết luận.

❷ Q2 – DLL được nạp ở địa chỉ khác

Đây có thể là hiện tượng relocation hoặc ASLR hợp lệ. Cần kiểm tra relocation entries, module base thực tế, RVA, kiến trúc, hash/chữ ký và đường dẫn DLL. Không kết luận DLL bị thay thế chỉ từ địa chỉ khác.

❸ Q3 – Nền tảng cho Bài 3

PE-loader-memory map tạo baseline của chương trình bình thường. Khi học lây nhiễm, sinh viên dùng baseline này để đối chiếu section, entry point, module, địa chỉ, quyền và luồng thực thi bị thay đổi như thế nào.

Tự học và chuẩn bị Lab 1

- Đọc lại L02 và hoàn thiện sơ đồ file–memory.
- Ôn địa chỉ, bộ nhớ ảo, process/thread và quyền R/W/X.
- Chuẩn bị máy ảo cô lập, snapshot và công cụ theo hướng dẫn.
- Chỉ dùng mẫu do giảng viên cung cấp; ghi hash, cấu hình và nguồn bằng chứng.

Câu hỏi chuyển tiếp

Nếu đã biết cấu trúc PE và quá trình loader, cần so sánh những gì để xác định tệp hoặc tiến trình đã bị mã độc can thiệp?

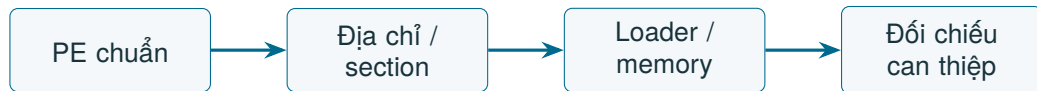
Thiết kế bằng chứng cho Lab 1

- File: hash, headers, sections, imports, TLS.
- Memory: image base, module list, section permissions.
- Runtime: process/thread và sự kiện loader.
- Tái lập: công cụ, phiên bản, cấu hình, snapshot, thời điểm.

Sản phẩm

Không chỉ ảnh chụp; mỗi ảnh phải giải thích được nhận định nào.

Bài 2 chuẩn bị gì cho Bài 3?



Thông điệp

Bài 2 xây dựng baseline; Bài 3 dùng baseline để giải thích tệp hoặc tiến trình đã bị thay đổi như thế nào.